



EASY PHENOTYPING LAB · SCIENTIFIC SOFTWARE

Canopy CCGR for Python

Version 1.0.0

A practical image-analysis tool for quantitative canopy coverage and canopy greenness

Abbas Haghshenas

Easy Phenotyping Lab (EPL)

haqueshenas.github.io/EPL

haqueshenas@gmail.com

Companion user guide

Scientific workflow · Segmentation · GUI use · Outputs · Reproducibility · Custom algorithms



Contents

This guide is designed for both crop-science researchers and technically experienced users. The PDF uses the same approved documentation text as the project README, with a print-oriented layout.

From the MATLAB Canopy CCGR capsule to a Python-based application	4
1. What is Canopy CCGR?	4
2. Why use image-based canopy measurements?	5
3. What do CC, GR, and CCGR mean?	6
4. What does “segmentation” mean?	7
5. The scientific workflow	7
6. Segmentation methods	8
7. The original G>R method	9
8. Alternative RGB methods	9
9. HSV segmentation	9
10. Live segmentation preview	10
11. Understanding the graphs	11
12. What is GRmax?	11
13. What happens when GR is outside the visualization range?	12
14. Choosing GRmax for a collection of images	13
15. Input images	13
16. Selecting the input and output folders	14
17. Output files	15
18. Segmented image outputs	16
19. Individual graphs	16
20. Comparison overview	17
21. Custom segmentation	17
22. Custom segmentation input	17
23. Custom segmentation output	18
24. Practical advice for Custom algorithms	19



25. Do I need to know Python?	19
26. Running from source	20
27. Scientific reproducibility	20
28. Important scientific considerations	21
29. Original scientific publication	21
30. Historical MATLAB software	22
31. Recommended citation for Canopy CCGR for Python v1.0.0	22
32. Easy Phenotyping Lab (EPL)	22
33. Software authorship and development	23
34. License	23
35. Final note to users	23



From the MATLAB Canopy CCGR capsule to a Python-based application

Canopy CCGR for Python v1.0.0 is the **Python reimplementations and extension of the Canopy CCGR computational workflow previously shared on Code Ocean as the MATLAB Version 2 capsule**:

Code Ocean - Canopy CCGR, MATLAB v2

<https://codeocean.com/capsule/2177070/tree/v2>

The earlier MATLAB implementation established the scientific workflow for separating canopy vegetation from background pixels, calculating RGB statistics, deriving the CC, GR, and CCGR indices, and producing segmented images and graphical representations.

The present release carries that established scientific concept and calculation path into a new Python implementation. It is therefore deliberately numbered **Version 1.0.0**, rather than “Version 3”: it is a new Python software line and not simply a continuation of the MATLAB version numbering.

The historical development can be summarized as:

MATLAB Canopy CCGR v1 -> MATLAB Canopy CCGR v2 -> Canopy CCGR for Python v1.0.0

The Python version also extends the original workflow with a graphical user interface, live segmentation preview, multiple segmentation methods, configurable HSV segmentation, user-defined Custom segmentation, improved visualization, vector graph export, and a Windows one-file executable.

1. What is Canopy CCGR?

Canopy CCGR is a research-oriented image-analysis application developed for researchers in crop science, agronomy, plant physiology, crop production, plant breeding, phenotyping, and related disciplines.

The software analyzes digital images of crop canopies by separating image pixels into two broad groups:

- **Vegetation**
- **Non-vegetation**, such as soil or background

It then uses the selected vegetation pixels to calculate three central quantities:

- **CC - Canopy Coverage**
- **GR - Green-Red contrast**
- **CCGR - the combined CC x GR index**

The purpose is not simply to produce a visually attractive segmented image. The main objective is to convert canopy images into quantitative measurements that can be compared across images, dates, treatments, genotypes, environments, irrigation regimes, stress conditions, or other experimental factors.



Canopy CCGR was originally developed in connection with research on image-based tracking of crop development and ripening and was designed as a quantitative complement to conventional phenological observations.

2. Why use image-based canopy measurements?

Traditional crop phenology relies on observations such as emergence, tillering, heading, flowering, grain filling, and maturity. These observations are scientifically valuable, but they are often based on visual judgments and discrete developmental stages.

Digital images provide another kind of information.

Instead of asking only:

“Has this canopy reached a particular stage?”

an image-analysis approach can ask:

“How much of the image is occupied by vegetation, and how strongly green are the selected vegetation pixels?”

This can provide a continuous quantitative description of canopy change over time.

Canopy CCGR can therefore be useful for studying phenomena such as:

- canopy development and senescence;
- ripening trends;
- effects of irrigation and water availability;
- soil-moisture-related responses;
- environmental stress;
- cold or heat effects;
- pest or disease-related canopy changes;
- genotype or cultivar differences;
- cultivar mixtures;
- image-based phenological monitoring.

The indices should nevertheless be interpreted in the context of the biological experiment. They are **image-derived quantitative traits**, not direct replacements for every physiological, biochemical, or agronomic measurement.



3. What do CC, GR, and CCGR mean?

3.1 CC - Canopy Coverage

CC describes the fraction of the image occupied by pixels classified as vegetation.

In simple terms:

CC answers the question: “How much of this image is vegetation?”

If 70% of the image pixels are classified as vegetation:

CC = 0.70

If 25% are vegetation:

CC = 0.25

For an image with a binary vegetation mask:

CC = Num_Veg / Num_Total

where:

- Num_Veg = number of vegetation pixels;
- Num_Total = total number of pixels in the image.

CC is therefore a coverage measure. It does not, by itself, tell us how green the selected vegetation pixels are.

3.2 GR - Green-Red contrast

GR describes the difference between the average green and red components of the selected vegetation pixels, relative to the average green component.

The established calculation is:

Rveg = sum(R x mask) / Temp

Gveg = sum(G x mask) / Temp

and:

GR = (Gveg - Rveg) / Gveg

In practical terms:

GR describes how strongly the selected vegetation pixels are shifted toward green relative to red.

Higher GR generally corresponds to stronger green-red contrast among the selected vegetation pixels.

Importantly, GR is calculated from the selected vegetation pixels. The segmentation method therefore determines which pixels contribute to the measurement.

3.3 CCGR - combining coverage and greenness

CCGR combines the two components:

CCGR = CC x GR



This means that CCGR reflects both:

1. how much vegetation occupies the image; and
2. how strongly the selected vegetation pixels express green-red contrast.

A canopy can therefore have substantial vegetation coverage while becoming progressively less green, or it can become greener while its coverage changes in another direction. CCGR combines these dimensions into a single derived quantity.

For interpretation and statistical analysis, CC, GR, and CCGR should generally be retained separately rather than treating CCGR as a replacement for the two underlying measures.

4. What does “segmentation” mean?

The term **segmentation** may sound complicated, but the basic idea is simple.

A digital image is made of pixels. Each pixel contains color information. Canopy CCGR examines those pixels and decides whether each one belongs to the vegetation segment or the non-vegetation segment.

The result is a **binary mask**:

- 1 = vegetation
- 0 = non-vegetation

For example, imagine an image containing:

- green wheat leaves;
- brown soil;
- shadows;
- residue;
- and other background objects.

A segmentation rule examines the color information of every pixel and creates a map indicating which pixels should be treated as vegetation.

That map is then used by the scientific calculation.

Segmentation is therefore not the same thing as measuring CC or GR. It is the step that determines **which pixels are allowed to contribute to those measurements**.

This is one of the most important concepts in Canopy CCGR.

5. The scientific workflow

The software follows a simple sequence:

Original image

->



Segmentation

->

Binary vegetation mask

->

RGB statistics of the selected pixels

->

CC and GR

->

CCGR

->

Results.csv + segmented images + graphs

A particularly important design principle in the Python implementation is that segmentation is performed on the **original full-resolution image first**.

Only after the segmentation mask has been created are images resized for display in the GUI.

This is intentional. Resizing an image before threshold-based segmentation can change pixel values and consequently change the resulting mask. Display resizing is therefore kept outside the scientific measurement path.

The scientific engine is the authoritative route for the calculations; the graphical interface does not implement a second independent CC/GR/CCGR calculation.

6. Segmentation methods

Canopy CCGR for Python provides five segmentation routes.

Method	Rule / parameters	General purpose
G>R	$G - R > 0$	Original MATLAB-compatible method
G>R&G>B	$G > R$ and $G > B$	More restrictive green selection
2G-R-B>0	$2G - R - B > 0$	Alternative RGB green-contrast rule
HSV	User-defined H, S, and V limits	Adjustable color-space segmentation
Custom	User-supplied Python <code>create_mask()</code>	Research-specific segmentation algorithms

The first three methods operate directly on RGB information.



7. The original G>R method

The default method is:

$$\mathbf{G - R > 0}$$

or, equivalently:

$$\mathbf{G > R}$$

A pixel is therefore classified as vegetation when its green value is greater than its red value.

This is the core segmentation rule used by the established MATLAB implementation and is retained as the default in the Python version.

For researchers reproducing results from the earlier MATLAB workflow, `G>R` should normally be the starting point.

8. Alternative RGB methods

$$\mathbf{G > R \& G > B}$$

This method requires both:

$$\mathbf{G > R}$$

and

$$\mathbf{G > B}$$

Compared with `G>R`, this is more restrictive because a pixel must be stronger in green than in both of the other RGB channels.

$$\mathbf{2G - R - B > 0}$$

This method uses:

$$\mathbf{2G - R - B > 0}$$

It provides another way to identify pixels with relatively strong green contribution.

These methods are alternatives, not interchangeable definitions. Changing the segmentation rule changes which pixels are selected and can therefore change CC, GR, and CCGR.

For comparative scientific work, the method used should always be reported.

9. HSV segmentation

HSV describes color using three components:

- **H - Hue**



- **S - Saturation**
- **V - Value**

HSV can be useful when simple RGB rules are not sufficiently selective for a particular image collection.

Canopy CCGR for Python allows the researcher to define both minimum and maximum values:

- `H_min`
- `H_max`
- `S_min`
- `S_max`
- `V_min`
- `V_max`

The default values are:

- **H: 25-95**
- **S: 40-255**
- **V: 30-255**

Hue uses the OpenCV-style range **0-179**, while Saturation and Value use **0-255**.

The GUI checks that each minimum does not exceed its corresponding maximum and provides a button to restore the default HSV values.

HSV settings are part of the scientific configuration of an analysis. They should therefore be recorded whenever HSV segmentation is used.

10. Live segmentation preview

Before processing an image collection, the application provides a live visual preview.

The preview shows three synchronized views:

Original

The source image as supplied to the analysis.

Vegetation

The pixels classified as vegetation remain visible, while the non-vegetation portion is displayed against a purple background.

Non-vegetation

The non-vegetation pixels remain visible, while the vegetation portion is covered by black.

The purpose of this preview is simple:

Look at the segmentation before trusting the resulting numbers.



A segmentation method can produce mathematically valid output while still being biologically inappropriate for a particular image collection. Visual inspection is therefore an important scientific quality-control step.

The preview uses the same segmentation route as the scientific engine rather than a separate simplified preview algorithm.

11. Understanding the graphs

Canopy CCGR produces a graphical representation of the calculated values for individual images.

The graph is designed to make the relationship among CC, GR, and CCGR visually intuitive.

Left panel - CC + GR

The outer black circle represents the **total image area**.

Inside it is a green circle representing the vegetation component.

The **area of the green circle corresponds to CC**.

In other words:

- larger green circle -> greater canopy coverage;
- smaller green circle -> lower canopy coverage.

The **color intensity** of the green circle represents the GR value after visualization normalization.

The numerical values are also printed beneath the graph.

The circle is therefore a visualization of the calculated result; it is not intended to reproduce the actual geometric shape of the canopy in the photograph.

Right panel - CCGR

The second panel uses a fixed-size circle.

Its green intensity represents the CCGR value after visualization normalization.

This allows researchers to compare the combined CC x GR result visually across images.

The graph therefore communicates two related ideas:

How much canopy is present and how green it is in the first panel, and
the combined CCGR response in the second.

The graph is a visualization of already calculated scientific values; it does not redefine those values.

12. What is GRmax?

GRmax is one of the most important settings to understand correctly.

GRmax is a graph-visualization parameter. It is not a scientific calculation parameter.

It controls the scale used to convert GR and related graph values into color intensity.

The default value is:

GRmax = 0.3

The original MATLAB workflow used this value based on empirical observations from a large collection of wheat canopy images collected during two growing seasons.

However, a critical point is:

GRmax does not change the measured GR, CC, or CCGR values.

It changes only how those values are represented visually.

This means that changing GRmax does not constitute recalculating the scientific results.

13. What happens when GR is outside the visualization range?

Suppose the analysis produces:

GR = 0.34

while:

GRmax = 0.30

The numerical value remains:

GR = 0.34

The software does not replace it with 0.30.

Instead, it recognizes that the value lies outside the current graph-normalization range and can generate a graph/display warning.

This distinction is deliberate:

measurement != visualization

The same principle applies to negative GR warnings.

With the original `G>R` method, a negative GR is unexpected because every selected vegetation pixel satisfies `G > R`; such a result deserves investigation.

With alternative segmentation methods, negative GR can occur because the selected pixels are not necessarily constrained by `G > R`. A negative value under those methods does not automatically mean that the calculation is wrong.

14. Choosing GRmax for a collection of images

When graphs from several images are intended to be compared with one another, a **common GRmax should be used for the entire image collection**.

If one or more images have GR values greater than the default 0.3:

1. open `Results.csv` ;
2. inspect the GR column;
3. identify the maximum GR in the collection;
4. use a GRmax at least as large as that maximum for the visualization;
5. regenerate the graphs if necessary.

For example, if the maximum measured GR is 0.37, use a visualization scale covering at least 0.37.

The important principle is consistency:

Do not compare graph colors produced with different GRmax values as though they represented the same visual scale.

Again, changing GRmax does not change the underlying numerical measurements.

15. Input images

The principal input is a folder containing canopy images.

For maximum compatibility and reproducibility, researchers are encouraged to use ordinary RGB raster images such as:

- `.jpg`
- `.jpeg`
- `.png`

The original MATLAB workflow specifically documented JPG canopy images.

For scientific experiments, consistent image acquisition is often more important than the particular filename extension.

Researchers should therefore try to keep the following conditions consistent when possible:

- camera and lens;
- camera angle;
- shooting distance;
- field of view;
- lighting conditions;
- exposure;
- time of day;
- background;
- canopy orientation;



- image resolution.

The software analyzes the image that it receives. It does not automatically know whether a change in image-derived greenness is caused by plant biology, illumination, camera settings, shadow, background effects, or another factor.

16. Selecting the input and output folders

The Python GUI is designed so that a user does not need to know Python to perform an ordinary analysis.

The basic workflow is:

Step 1 - Launch

Run the Windows executable, or run `main.py` when using the source code.

Step 2 - Select the segmentation method

Choose:

- `G>R`
- `G>R&G>B`
- `2G-R-B>0`
- `HSV`
- `Custom`

Step 3 - Configure the method

For HSV, enter the desired H, S, and V limits.

For Custom, select the required `.py` segmentation file.

Step 4 - Select the image folder

Choose the folder containing the canopy images.

Step 5 - Select the output folder

Choose where the analysis results should be written.

Step 6 - Inspect the preview

Review the Original, Vegetation, and Non-vegetation panels.

Step 7 - Start processing

Press:

START PROCESSING

Step 8 - Review the results

Inspect:

- `Results.csv` ;
- segmented image outputs;
- individual graphs;
- comparison output;
- any warnings.

This workflow is intentionally simple enough for users whose background is crop science rather than computer science.

17. Output files

The principal output is:

Results.csv

This is the numerical results table generated by the scientific processing engine.

It contains the image information, pixel counts, RGB statistics, and the derived CC, GR, and CCGR measurements.

The RGB-related statistics follow the established Canopy CCGR structure and distinguish among vegetation, non-vegetation, and overall image information.

The terminology used in the original workflow includes:

- `Num` - number of pixels;
- `Num_Total` - total image pixels;
- `Red` , `Green` , and `Blue` - RGB channels;
- `Mean` - average value;
- `Var` - variance;
- `Veg` - vegetation segment;
- `Soil` / non-vegetation - background segment;
- `Overall` - entire image.

The exact CSV structure should be retained when comparing the Python implementation with earlier Canopy CCGR studies.

MATLAB-Python output differences

Users comparing a complete `Results.csv` file from the Python implementation with results previously generated by the MATLAB version may notice differences in some secondary statistical fields, particularly **variance-related values**.

This does not necessarily indicate an error in either implementation.

One important source of such differences is that MATLAB and Python's numerical/statistical functions can use different default conventions. In particular, MATLAB's `var(A)` uses **N-1 normalization by default**, whereas NumPy's `var()` uses **N normalization by default (ddof=0)**. Thus, even when the same underlying pixel values are supplied, corresponding variance values can differ when the two implementations use their respective defaults.

This is therefore not simply a matter of “MATLAB versus Python” or one library being more accurate. It is, in part, a difference in the **statistical definition used by the corresponding function by default**. Additional very small differences may also arise from floating-point representation, numerical implementation, and the order of arithmetic operations.

These possible differences should be considered when comparing detailed RGB variance fields between the MATLAB and Python versions. The established scientific calculation path for the principal Canopy CCGR metrics is retained in the Python implementation, and visualization parameters such as `GRmax` do not alter those numerical metrics.

18. Segmented image outputs

The analysis also produces visual representations of the segmentation.

Two basic concepts are retained from the original workflow:

Vegetation image

Vegetation remains visible while non-vegetation is represented by a purple background.

Non-vegetation image

Non-vegetation remains visible while vegetation is covered by black.

These images are useful for visually checking whether the segmentation corresponds to the intended biological interpretation.

They should not be confused with the original photographs; they are derived visualization products.

19. Individual graphs

For processed images, Canopy CCGR generates graphical representations of the numerical results.

Graph outputs are provided as:

- **PDF** - vector-quality output suitable for scientific documents and publication workflows;
- **PNG** - convenient raster output for viewing, presentations, or sharing.

The individual graph contains the CC + GR panel and the CCGR panel described earlier.

The current visualization engine also keeps the scientific numbers separate from graph normalization and produces a warning graph when the GR value lies outside the selected visualization range.



20. Comparison overview

The software can also generate a comparison overview containing the graphical summaries of multiple analyzed images.

This provides a convenient way to inspect the progression or variation of CC, GR, and CCGR across an image collection.

The comparison output is generated from the already-calculated results. It does not create a second scientific calculation.

This is particularly useful when analyzing:

- repeated measurements over time;
 - multiple cultivars;
 - irrigation treatments;
 - stress treatments;
 - field experiments;
 - genotype x environment combinations.
-

21. Custom segmentation

The **Custom** method is intended for researchers who need a segmentation algorithm that is not covered by the built-in RGB or HSV rules.

A Custom segmentation method is simply a Python file containing a function called:

```
def create_mask(image):  
    # create and return a binary vegetation mask  
    return mask
```

The interface is intentionally small.

The user does not need to modify the Canopy CCGR GUI or scientific engine.

22. Custom segmentation input

The function receives the original image as a NumPy array:

```
Type:      NumPy ndarray  
dtype:     uint8  
shape:     H x W x 3  
format:    RGB  
resolution: original full resolution
```

The three channels correspond to:



R, G, B

The Custom function receives a copy of the original image, so the custom algorithm cannot alter the source array used by the remainder of the scientific pipeline.

23. Custom segmentation output

The function must return a two-dimensional mask:

```
shape: H x W
```

with only binary values:

```
0 = non-vegetation  
1 = vegetation
```

Boolean output is also accepted.

For example:

```
import numpy as np  
  
def create_mask(image):  
    # Example only.  
    # Replace this with your own segmentation algorithm.  
  
    r = image[:, :, 0]  
    g = image[:, :, 1]  
    b = image[:, :, 2]  
  
    mask = (  
        (g > r)  
        & (g > b)  
    )  
  
    return mask
```

The following are **not** accepted as equivalent binary output:

```
0 / 255
```

The researcher should convert such masks to:

```
0 / 1
```

before returning them.



The application validates the returned object and rejects inappropriate dimensions, non-array outputs, NaN/Inf values, complex values, and non-binary values.

24. Practical advice for Custom algorithms

A Custom algorithm should be tested independently before being used for a large experiment.

At minimum, verify:

1. the input image has the expected RGB format;
2. the returned mask has exactly the same height and width;
3. the mask contains only 0/1 or Boolean values;
4. the segmentation visually identifies the intended vegetation;
5. the algorithm behaves consistently on representative images.

For reproducibility, preserve:

- the Custom `.py` file;
- the algorithm version;
- its parameters;
- the Canopy CCGR software version;
- the original images;
- the resulting `Results.csv`.

A Custom segmentation file is part of the scientific configuration of the analysis and should therefore be archived with the resulting dataset.

25. Do I need to know Python?

No - not for ordinary use.

The easiest way to use Canopy CCGR for Python on Windows is the packaged executable.

A researcher can:

1. launch the application;
2. select the image folder;
3. select the output folder;
4. select a segmentation method;
5. inspect the preview;
6. start processing;
7. inspect the results.

Python knowledge becomes relevant mainly when the researcher wants to:

- run the program from source;

- develop a Custom segmentation algorithm;
- modify or extend the software;
- reproduce the build environment.

The source code is nevertheless distributed openly so that technically experienced users can inspect, reproduce, test, modify, and extend the implementation.

26. Running from source

Users who prefer to work with the source code can run Canopy CCGR directly in Python.

The repository provides the required source files and dependency information.

A typical workflow is:

```
Clone or download the repository
->
Install Python
->
Install the listed dependencies
->
Run main.py
```

The Windows executable is provided as a convenience for users who do not want to install or manage the Python environment.

27. Scientific reproducibility

For scientific work, the following information should be preserved with every analysis:

- Canopy CCGR version;
- segmentation method;
- HSV parameters, when applicable;
- Custom segmentation file, when applicable;
- GRmax used for graph visualization;
- original image collection;
- `Results.csv` ;
- exported graphs;
- any relevant acquisition information.

For example, a reproducible record might state:



```
Software:      Canopy CCGR for Python v1.0.0
Segmentation: HSV
H:            25-95
S:            40-255
V:            30-255
GRmax:        0.30
```

This makes it easier to determine whether differences among analyses arise from biological variation, image acquisition, segmentation settings, or software changes.

28. Important scientific considerations

Canopy CCGR is an image-analysis tool, not an automatic substitute for experimental interpretation.

Segmentation quality is fundamental because the selected pixels determine the subsequent calculations.

Researchers should therefore be cautious when images contain:

- strong shadows;
- very unusual illumination;
- reflective surfaces;
- colored soil;
- residues or weeds;
- senescent leaves;
- strongly yellow or brown vegetation;
- objects with colors similar to vegetation;
- large changes in camera conditions.

A method that works well for one image collection may not necessarily be optimal for another.

The software therefore exposes multiple segmentation methods rather than assuming that a single rule is universally appropriate.

29. Original scientific publication

The Canopy CCGR methodology is associated with the following scientific article:

Haghshenas, A., & Emam, Y. (2019).

Image-based tracking of ripening in wheat cultivar mixtures: A quantifying approach parallel to the conventional phenology.

Computers and Electronics in Agriculture, 156, 318-333.

DOI:

<https://doi.org/10.1016/j.compag.2018.11.020>



The 2019 publication provides the scientific context and methodological foundation for the original Canopy CCGR work.

Researchers using the software for scientific publications are encouraged to cite the relevant article as well as the specific software version when appropriate.

30. Historical MATLAB software

The original MATLAB Version 2 documentation identified the following historical authors of the software:

- **Abbas Haghshenas**
- **Yahya Emam**
- **Saeid Jafarizadeh**

That historical software represents the earlier stage of the Canopy CCGR project.

The present Python release is a new implementation led by Abbas Haghshenas and should be cited as the Python software release when the Python implementation itself is used.

31. Recommended citation for Canopy CCGR for Python v1.0.0

Haghshenas, A. (2026). *Canopy CCGR for Python (Version 1.0.0)*. Easy Phenotyping Lab (EPL).

Once the software has been archived in Zenodo, the version-specific DOI should be added to this citation.

32. Easy Phenotyping Lab (EPL)

Canopy CCGR is developed and shared through the **Easy Phenotyping Lab (EPL)**.

The Easy Phenotyping Lab is a scientific initiative dedicated to simplifying and democratizing crop phenotyping for researchers worldwide.

EPL develops simple, novel, and reliable solutions for common phenotyping challenges in crop and plant sciences. It also shares software tools, code snippets, computational models, and new measurement approaches intended to make practical crop phenotyping more accessible.

Easy Phenotyping Lab (EPL)

<https://haqueshenas.github.io/EPL>

Initiated by:



Abbas Haghshenas

Independent Researcher in Crop Production, Shiraz, Iran

Contact:

haqueshenas@gmail.com

33. Software authorship and development

The software concept, scientific design, methodological decisions, project direction, and overall development were led by **Abbas Haghshenas**; the Python code was developed with coding assistance from **OpenAI's GPT-5.6 Luna**.

34. License

Canopy CCGR for Python v1.0.0 is released under the:

MIT License

Copyright (c) 2026 Abbas Haghshenas

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

The same license text is provided separately in the repository as:

LICENSE

35. Final note to users

Canopy CCGR was created around a simple idea:



A crop image contains quantitative information that can be measured, not merely viewed.

The software is intended to make that information more accessible to researchers in crop science while keeping the scientific calculation path transparent.

For an ordinary user, the workflow is straightforward:

Choose images -> choose segmentation -> inspect preview -> process -> examine Results.csv and graphs.

For an experienced researcher, the same system provides access to the source code, segmentation rules, configurable HSV parameters, Custom algorithms, reproducible outputs, and versioned releases.

The goal is not to make phenotyping more complicated.

It is to make useful quantitative phenotyping simpler, more transparent, more reproducible, and more accessible.

Project links

Easy Phenotyping Lab (EPL)

<https://haqueshenas.github.io/EPL>

Historical MATLAB Canopy CCGR v2 capsule on Code Ocean

<https://codeocean.com/capsule/2177070/tree/v2>

Scientific article

<https://doi.org/10.1016/j.compag.2018.11.020>

Contact

haqueshenas@gmail.com

Canopy CCGR for Python v1.0.0

Abbas Haghshenas - Easy Phenotyping Lab (EPL)

Copyright (c) 2026 Abbas Haghshenas

MIT License



Canopy CCGR for Python v1.0.0

Abbas Haghshenas - Easy Phenotyping Lab (EPL)

Copyright (c) 2026 Abbas Haghshenas · MIT License